***Ejercicios Tema 5. Diseño y realización de pruebas con JUnit***

## Objetivos

1. Analizar código ya implementado.
2. Calcular la complejidad ciclomática de métodos sencillos.
3. Diseñar casos de prueba que cubran todas las ramas.
4. Implementar pruebas unitarias con JUnit.
5. Diseñar pequeños programas propios y justificar sus pruebas a partir de la complejidad del código.

## Consideraciones

En **todos los ejercicios** debes entregar:

1. **Análisis del método o programa**
   - Qué hace.
   - Qué entradas admite.
   - Qué salidas produce.
   - Casos normales y casos límite.
2. **Complejidad ciclomática**
   - Cálculo razonado del número mínimo de tests para cubrir las decisiones.
3. **Diseño de casos de prueba**

   Para cada caso:
   - ID
   - Descripción
   - Entrada
   - Resultado esperado
   - Oráculo
   - Rama o camino que cubre
4. **Implementación en JUnit**
   - Clase `...Test` en `src/test/java`
   - Métodos de test bien nombrados

- Uso de `assertEquals`, `assertTrue`, `assertThrows`, `assertDoesNotThrow` cuando proceda.

## Plantilla para los casos de prueba en cada ejercicio

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | | | | |
| TC02 | | | | |
| TC03 | | | | |
| ... | | | | |

## Ejercicio 1. Método signo

Se proporciona el siguiente método

```
public static int signo(int x) {
 if (x > 0) {
 return 1;
 } else if (x < 0) {
 return -1;
 }
 return 0;
}
```

### Tareas

1. Calcula la complejidad ciclomática.
2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
3. Implementa la clase `SignoTest` con JUnit.

Solución:

1-Análisis del método

)Qué hace: Devuelve el signo de un número entero.

)Qué entradas admite: Un número entero.

)Qué salidas produce. Un valor positivo o negativo.

)Casos normales y casos límite. Normales: Positivo o negativo y límite : 0.

2-Complejidad ciclomática

Numero de decisiones =2 ( if (x > 0)) , (else if (x < 0)

Minimo de pruebas necesarias = 2+1=3

3-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Número positivo | 5 | 1 | OK |
| TC02 | Número negativo | -5 | -1 | OK |
| TC03 | 0 | 0 | 0 | OK |

4-Implementación en JUnit

Ejercicio1.java | SignoTest.java

```java

package com.joanvaldes.tema4gradle;
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class SignoTest {

    @Test
    void testPositivo() {
        assertEquals( expected: 1, Ejercicio1.signo( x: 10));
    }

    @Test
    void testNegativo() {
        assertEquals( expected: -1, Ejercicio1.signo( x: -5));
    }

    @Test
    void testCero() {
        assertEquals( expected: 0, Ejercicio1.signo( x: 0));
    }
}

```

## Ejercicio 2. Método clasificarEdad

Se proporciona el siguiente método:

```java
public static String clasificarEdad(int edad) {
 if (edad < 0) {
 throw new IllegalArgumentException("Edad no válida");  }

 if (edad < 6) {
 return "Infancia";
 } else if (edad < 12) {
 return "Niñez";
 } else if (edad < 18) {
 return "Adolescencia";
```

```
} else if (edad < 25) {
return "Juventud";
} else if (edad < 60) {
return "Adultez";
} else {
return "Vejez";
}
}
```

**Tareas**

1. Calcula la complejidad ciclomática.
2. Diseña el conjunto mínimo de pruebas para cubrir todas las ramas.
3. Añade además casos límite especialmente relevantes.
4. Implementa `ClasificarEdadTest`.

Solución:

1-Análisis del método

)Qué hace: Clasifica una edad en una etapa de vida y si la edad es negativa lanza una excepción

)Qué entradas admite: Un número positivo

)Qué salidas produce: la etapa correspondiente o una excepción si la edad es negativa

)Casos normales y casos límite. Normales: 0 al 100 aprox y límite : edad negativa y cambios de estapa: 0, 5, 6, 11, 12, 17, 18, 24, 25, 59, 60.

2-Complejidad ciclomática

1. if (edad < 0)
2. if (edad < 6)
3. else if (edad < 12)
4. else if (edad < 18)
5. else if (edad < 25)
6. else if (edad < 60)

Total decisiones = 6
Mínimo de pruebas necesarias = 6 + 1 = 7

3-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Edad negativa | -1 | Excepción | OK |
| TC02 | Infancia inicio | 0 | "Infancia" | OK |
| TC03 | Infancia fin | 5 | "Infancia" | OK |
| TC04 | Niñez | 6 | "Niñez" | OK |
| TC05 | Adolescencia | 12 | "Adolescencia" | OK |
| TC06 | Juventud | 18 | "Juventud" | OK |
| TC07 | Adultez | 25 | "Adultez" | OK |
| TC08 | Vejez | 60 | "Vejez" | OK |

4-Implementación en JUnit

```java
public class ClasificarEdadTest {

        @Test
        void testEdadNegativa() {
            assertThrows(IllegalArgumentException.class, () -> Ejercicio2.clasificarEdad(-1));
        }

        @Test
        void testInfancia() {
            assertEquals( expected: "Infancia", Ejercicio2.clasificarEdad(0));
            assertEquals( expected: "Infancia", Ejercicio2.clasificarEdad(5));
        }

        @Test
        void testNinez() {
            assertEquals( expected: "Niñez", Ejercicio2.clasificarEdad(6));
            assertEquals( expected: "Niñez", Ejercicio2.clasificarEdad(11));
        }

        @Test
        void testAdolescencia() {
            assertEquals( expected: "Adolescencia", Ejercicio2.clasificarEdad(12));
            assertEquals( expected: "Adolescencia", Ejercicio2.clasificarEdad(17));
        }

        @Test
        void testJuventud() {
            assertEquals( expected: "Juventud", Ejercicio2.clasificarEdad(18));
            assertEquals( expected: "Juventud", Ejercicio2.clasificarEdad(24));
        }

        @Test
        void testAdultez() {
            assertEquals( expected: "Adultez", Ejercicio2.clasificarEdad(25));
            assertEquals( expected: "Adultez", Ejercicio2.clasificarEdad(59));
        }

        @Test
        void testVejez() {
            assertEquals( expected: "Vejez", Ejercicio2.clasificarEdad(60));
            assertEquals( expected: "Vejez", Ejercicio2.clasificarEdad(90));
        }
    }
```

## Ejercicio 3. Método contarPositivos

Se proporciona el siguiente método:

```
public static int contarPositivos(int[] datos) {
 int contador = 0;
 for (int i = 0; i < datos.length; i++) {
 if (datos[i] > 0) {
 contador++;
 }
 }
 return contador;
}
```

**Tareas**

1. Calcula la complejidad ciclomática.
2. Diseña pruebas para cubrir:
   - array vacío,
   - array con todos negativos,
   - array con mezcla de positivos, negativos y ceros.
3. Justifica qué ramas cubre cada prueba.
4. Implementa `ContarPositivosTest`.

Solución:

1-Análisis del método

)Qué hace: Cuenta cuántos números positivos hay dentro de un array de enteros.

)Qué entradas admite: Un array de números enteros (`int[]`). o un número negativo, positivo o 0.

)Qué salidas produce:Un contador que es un número entero que indica el número de valores positivos contiene el array.

)Casos normales y casos límite. Normales: array con valores combinados y límite : Array vacío, array con todos negativos, array con ceros, array con un elemento, array con todos positivos

2-Complejidad ciclomática

Total decisiones = 2
Mínimo de pruebas necesarias = 2+1=3

3-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Array vacío | [] | 0 | OK |
| TC02 | Array con todos negativos | [-3, -1, -7] | 0 | OK |
| TC03 | positivos, negativos y ceros | [-1, 0, 4, 7, -3] | 2 | OK |

TC01 – Array vacío

- El for no entra
- El if no se evalúa

TC02 – Array con todos negativos

- El for se ejecuta
- El if siempre es false

TC03 – Mezcla

- El for se ejecuta
- El if es true a veces
- También cubre if false en negativos y ceros.

4-Implementación en JUnit

Ejercicio3.java | ContarPositivosTest.java

```java
package com.joanvaldes.tema4gradle;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class ContarPositivosTest {

    @Test
    void testArrayVacio() {
        int[] datos = {};
        assertEquals( expected: 0, Ejercicio3.contarPositivos(datos));
    }

    @Test
    void testTodosNegativos() {
        int[] datos = {-3, -1, -7};
        assertEquals( expected: 0, Ejercicio3.contarPositivos(datos));
    }

    @Test
    void testMezclaValores() {
        int[] datos = {-1, 0, 4, 7, -3};
        assertEquals( expected: 2, Ejercicio3.contarPositivos(datos));
    }
}
```

## Ejercicio 4. Método calificacion



Se proporciona el siguiente método:

```
public static String calificacion(int nota) {
 if (nota < 0 || nota > 10) {
 throw new IllegalArgumentException("Nota no válida");  }

 return switch (nota) {
 case 0, 1, 2, 3, 4 -> "SUSPENSO";
 case 5 -> "SUFICIENTE";
 case 6 -> "BIEN";
 case 7, 8 -> "NOTABLE";
 default -> "SOBRESALIENTE";
 };
}
```

**Tareas**

1. Calcula la complejidad ciclomática
2. Diseña los casos mínimos para cubrir todas las alternativas del `switch` y la validación inicial.
3. Implementa `CalificacionTest`.
4. Indica qué casos podrían agruparse con `@ParameterizedTest`.

1-Análisis del método
)Qué hace: Clasifica una nota del 0 a 10 en SUSPENSO, SUFICIENTE, BIEN, NOTABLE o SOBRESALIENTE.

)Qué entradas admite: un numero entero del 0 al 10 fuera de ese rango donde se lanzara un IllegalArgumentException

)Qué salidas produce:  SUSPENSO, SUFICIENTE, BIEN, NOTABLE,  SOBRESALIENTE o IllegalArgumentException

)Casos normales y casos límite. Normales:notas dentro del rango (0 a 10) que correspondan a las categorías. Límite: nota negativa o mayor que 10

2-Complejidad ciclomática
Total decisiones = 6
Mínimo de pruebas necesarias = 6+1=7

3-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Nota negativa | -1 | Excepción | OK |
| TC02 | Nota mayor que 10 | 11 | Excepción | OK |
| TC03 | Suspenso | 0 | Suspenso | OK |
| TC04 | Suficiente | 5 | Suficiente | OK |
| TC05 | Bien | 6 | Bien | OK |
| TC06 | Notable | 7 | Notable | OK |
| TC07 | Sobresaliente | 9 | Sobresaliente | OK |

4-Implementación en JUnit

Ejercicio4.java | CalificacionTest.java

```java
package com.joanvaldes.tema4gradle;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class CalificacionTest {

    @Test
    void testNotaNegativa() {
        assertThrows(IllegalArgumentException.class, () -> Ejercicio4.calificacion( nota: -1));
    }

    @Test
    void testNotaMayorQueDiez() {
        assertThrows(IllegalArgumentException.class, () -> Ejercicio4.calificacion( nota: 11));
    }

    @Test
    void testSuspenso() {
        assertEquals( expected: "SUSPENSO", Ejercicio4.calificacion( nota: 0));
        assertEquals( expected: "SUSPENSO", Ejercicio4.calificacion( nota: 4));
    }

    @Test
    void testSuficiente() {
        assertEquals( expected: "SUFICIENTE", Ejercicio4.calificacion( nota: 5));
    }

    @Test
    void testBien() {
        assertEquals( expected: "BIEN", Ejercicio4.calificacion( nota: 6));
    }

    @Test
    void testNotable() {
        assertEquals( expected: "NOTABLE", Ejercicio4.calificacion( nota: 7));
        assertEquals( expected: "NOTABLE", Ejercicio4.calificacion( nota: 8));
    }

    @Test
    void testSobresaliente() {
        assertEquals( expected: "SOBRESALIENTE", Ejercicio4.calificacion( nota: 9));
```

5. Indica qué casos podrían agruparse con `@ParameterizedTest`.
Grupo 1 – Suspenso
Valores: 0, 1, 2, 3, 4
Grupo 2 – Notable
Valores: 7, 8
Grupo 3 – Sobresaliente
Valores: 9, 10
Grupo 4 – Excepciones
Valores inválidos: -1, 11

## Ejercicio 5. Crear un método esBisiesto

Implementa un método con la siguiente firma:

```
public static boolean esBisiesto(int anyo)
```

### Reglas

Un año es bisiesto si:

- es divisible por 4 y no por 100, o
- es divisible por 400.

### Tareas

1. Implementa el método.
2. Calcula su complejidad ciclomática a partir de tu código.
3. Diseña los casos de prueba necesarios para cubrir todas las decisiones.
4. Implementa `EsBisiestoTest`.

### Requisitos

Debes incluir al menos estos tipos de caso:

- divisible por 4 y no por 100,
- divisible por 100 y no por 400,
- divisible por 400,
- no divisible por 4.

1-

```
package com.joanvaldes.tema4gradle;

public class Ejercicio5 {

    public static boolean esBisiesto(int anyo) {

        return (anyo % 4 == 0 && anyo % 100 != 0) || (anyo % 400 == 0);

    }

}
```

2-Análisis del método
)Qué hace: Determina si un año es bisiesto o no

)Qué entradas admite: Un número entero( positivo, negativo o cero).

)Qué salidas produce: un booleano, true si es bisiesto y false si no

)Casos normales y casos límite. Normales:años que cumplan o no las reglas de divisibilidad

Límite: Años divisibles por 4 pero no por 100

Años divisibles por 100 pero no por 400

Años divisibles por 400

Años no divisibles por 4

3-Complejidad ciclomática
Total decisiones = 3
Mínimo de pruebas necesarias = 3+1=4

4-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Divisible por 4 y no por 100 | 2024 | true | OK |
| TC02 | Divisible por 100 y no por 400 | 1900 | false | OK |
| TC03 | Divisible por 400 | 2000 | true | OK |
| TC04 | No divisible por 4 | 2023 | false | OK |

5-Implementación en JUnit

Ejercicio5.java | EsBisiestoTest.java

```java
package com.joanvaldes.tema4gradle;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class EsBisiestoTest {

    @Test
    void testDivisiblePor4NoPor100() {
        assertTrue(Ejercicio5.esBisiesto(anyo: 2024));
    }

    @Test
    void testDivisiblePor100NoPor400() {
        assertFalse(Ejercicio5.esBisiesto(anyo: 1900));
    }

    @Test
    void testDivisiblePor400() {
        assertTrue(Ejercicio5.esBisiesto(anyo: 2000));
    }

    @Test
    void testNoDivisiblePor4() {
        assertFalse(Ejercicio5.esBisiesto(anyo: 2023));
    }
}
```

## Ejercicio 6. Implementar Piedra – Papel – Tijera

Implementa el método:

public static String jugar(String jugador1, String jugador2)

Las jugadas posibles son:

- `"PIEDRA"`
- `"PAPEL"`
- `"TIJERA"`

**Reglas**

- Piedra gana a tijera
- Tijera gana a papel
- Papel gana a piedra
- Si ambas jugadas son iguales --> `"EMPATE"`
- Si alguna jugada no es válida --> `IllegalArgumentException`

**Tareas**

1. Implementar el método.
2. Calcular la complejidad ciclomática.
3. Diseñar los tests mínimos para cubrir todas las ramas.
4. Implementar `PiedraPapelTijeraTest`.

1-Implementar el método

```
package com.joanvaldes.tema4gradle;

public class Ejercicio6 {

    public static String jugar(String jugador1, String jugador2) {

        if (!esValida(jugador1) || !esValida(jugador2)) {

            throw new IllegalArgumentException("Jugada no válida");

        }

        if (jugador1.equals(jugador2)) {

            return "EMPATE";

        }

        if (jugador1.equals("PIEDRA") && jugador2.equals("TIJERA") ||

            jugador1.equals("TIJERA") && jugador2.equals("PAPEL") ||

            jugador1.equals("PAPEL") && jugador2.equals("PIEDRA")) {
```

```
            return "GANA JUGADOR 1";
        }
        return "GANA JUGADOR 2";
    }
    private static boolean esValida(String jugada) {
        return jugada.equals("PIEDRA") ||
               jugada.equals("PAPEL") ||
               jugada.equals("TIJERA");
    }
}
```

2-Análisis del método

)Qué hace: Determina el resultado de una partida de Piedra–Papel–Tijera

)Qué entradas admite: dos string que deben ser piedra papel  o tijera o una jugada no valida que daria excepcion

)Qué salidas produce: gana jugador1,gana jugador2,empate o excepcion

)Casos normales y casos límite. Normales:años que cumplan o no las reglas de divisibilidad

Límite: Años divisibles por 4 pero no por 100,años divisibles por 100 pero no por 400, años divisibles por 400,años no divisibles por 4

3-Complejidad ciclomática

Total decisiones = 3

Mínimo de pruebas necesarias = 3+1=4

4-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Jugada inválida | PIEDRA, PAPLEL | Excepción | OK |
| TC02 | Empate | PIEDRA,PIEDRA | Empate | OK |
| TC03 | Gana jugador 1 | PIEDRA,TIJERA | Gana jugador 1 | OK |
| TC04 | Gana jugador 2 | TIJERA,PIEDRA | Gana jugador 2 | OK |

5-Implementación en JUnit

Ejercicio6.java | PiedraPapelTijeraTest.java

```java
package com.joanvaldes.tema4gradle;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class PiedraPapelTijeraTest {

    @Test
    void testJugadaInvalida() {
        assertThrows(IllegalArgumentException.class,
                () -> Ejercicio6.jugar("PIEDRA", "LAGARTO"));
    }

    @Test
    void testEmpate() {
        assertEquals( expected: "EMPATE", Ejercicio6.jugar("PAPEL", "PAPEL"));
    }

    @Test
    void testGanaJugador1() {
        assertEquals( expected: "GANA JUGADOR 1", Ejercicio6.jugar("PIEDRA", "TIJERA"));
        assertEquals( expected: "GANA JUGADOR 1", Ejercicio6.jugar("TIJERA", "PAPEL"));
        assertEquals( expected: "GANA JUGADOR 1", Ejercicio6.jugar("PAPEL", "PIEDRA"));
    }

    @Test
    void testGanaJugador2() {
        assertEquals( expected: "GANA JUGADOR 2", Ejercicio6.jugar("TIJERA", "PIEDRA"));
        assertEquals( expected: "GANA JUGADOR 2", Ejercicio6.jugar("PAPEL", "TIJERA"));
        assertEquals( expected: "GANA JUGADOR 2", Ejercicio6.jugar("PIEDRA", "PAPEL"));
    }
}
```

## Ejercicio 7. Implementar Validador de contraseñas

Implementa el método:

public static int evaluarPassword(String password)

### Reglas

Se asignan puntos:

- +1 si tiene al menos **8 caracteres**
- +1 si tiene **mayúsculas**
- +1 si tiene **minúsculas**
- +1 si tiene **números**

- +1 si tiene **símbolos**

Interpretación del resultado:

- entre 0 y 2 --> "DEBIL"
- entre 3 y 4 --> "MEDIA"



- 5 --> "FUERTE"

**Tareas**

1. Implementar el método.
2. Calcular complejidad ciclomática.
3. Diseñar casos de prueba representativos.
4. Implementar `PasswordTest`.

1-Implementar el método

```
package com.joanvaldes.tema4gradle;

public class Ejercicio7 {

    public static int evaluarPassword(String password) {

        int puntos = 0;

        if (password.length() >= 8) puntos++;

        if (password.matches(".*[A-Z].*")) puntos++;

        if (password.matches(".*[a-z].*")) puntos++;

        if (password.matches(".*[0-9].*")) puntos++;

        if (password.matches(".*[^A-Za-z0-9].*")) puntos++;


        return puntos;

    }

}
```

2-Análisis del método

)Qué hace: Evalúa la fortaleza de una contraseña por puntos

)Qué entradas admite: un string password con cualquier tipo de caracteres

)Qué salidas produce: Un número entero entre 0 y 5 que representa la puntuación

)Casos normales y casos límite. Normales:contraseñas con combinaciones de letras, números y símbolos.

Límite: Contraseña vacía, contraseña sin mayúsculas, contraseña sin minúsculas, contraseña sin números, contraseña sin símbolos, contraseña con todos los tipos de caracteres

3-Complejidad ciclomática

Total decisiones = 5

Mínimo de pruebas necesarias = 5+1=6

4-Diseño de casos de prueba

| ID | Descripción | Entrada | Esperado | Estado |
|---|---|---|---|---|
| TC01 | Contraseña vacía | “” | 0 | OK |
| TC02 | Solo minúsculas | abcdefg" | 1 | OK |
| TC03 | Minúsculas + mayúsculas | "Abc12345" | 2 | OK |
| TC04 | Con números | "Abc12345" | 4 | OK |
| TC05 | Con símbolos | "Abc123$%" | 5 | OK |
| TC06 | Solo números | "12345678" | 2 | OK |

5-Implementación en JUnit

```java
package com.joanvaldes.tema4gradle;

import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

public class PasswordTest {

    @Test
    void testVacia() {
        assertEquals( expected: 0, Ejercicio7.evaluarPassword(""));
    }

    @Test
    void testSoloMinusculas() {
        assertEquals( expected: 1, Ejercicio7.evaluarPassword("abcdefg"));
    }

    @Test
    void testMayusculasYMinusculas() {
        assertEquals( expected: 2, Ejercicio7.evaluarPassword("Abcdefg"));
    }

    @Test
    void testConNumeros() {
        assertEquals( expected: 4, Ejercicio7.evaluarPassword("Abc12345"));
    }

    @Test
    void testConSimbolos() {
        assertEquals( expected: 5, Ejercicio7.evaluarPassword("Abc123$%"));
    }

    @Test
    void testSoloNumeros() {
        assertEquals( expected: 2, Ejercicio7.evaluarPassword("12345678"));
    }
}
```